

TerraFly Technical Terms - Complete Explanation Guide

Part 1: Individual Term Explanations

1. OPTICAL RGB IMAGE

What it is: A normal color photograph taken by satellites or aircraft. Contains three color channels - Red, Green, and Blue - just like photos from your phone camera.

In simple terms: This is the "pretty picture" you see - shows colors of buildings, roads, trees, water. But it's FLAT - no height information.

Example: A Google Maps satellite view of your city. You see buildings, roads, parks in color, but you can't tell how tall anything is.

2. REMOTE SENSING

What it is: Taking photographs or measurements of Earth from far away - satellites in space, aircraft flying high, or drones.

In simple terms: Like taking selfies, but from space or aircraft instead of holding a phone.

Why it matters: We can capture large areas quickly without sending people on the ground everywhere.

3. SINGLE-VIEW

What it is: Only ONE photograph from ONE angle. No second picture from a different position.

In simple terms: Like taking ONE photo of a building. You can't walk around it and take another from the side.

The challenge: With one photo, your brain can GUESS depth, but you don't KNOW the exact measurements. That's why Photo AI is hard.

4. DEPTH MAP

What it is: An AI's estimate of which parts of an image are closer to the camera and which are farther away.

In simple terms: The AI looks at shadows, textures, patterns and says "this probably looks close, that probably looks far."

Important: It's a GUESS based on learned patterns, not a measurement. Like your brain guessing distance in a photograph.

What it looks like: Often shown as a grayscale image - bright areas are close, dark areas are far (or vice versa).

5. ELEVATION

What it is: Height measured from a reference point - usually sea level (mean sea level = MSL).

In simple terms: How many meters UP from the ocean is this point?

Example: Mount Everest elevation = 8,849 meters above sea level. Your city might be 500 meters above sea level.

6. DEM (Digital Elevation Model)

What it is: A grid of elevation values representing the BARE GROUND terrain - hills, valleys, mountains. Trees and buildings are removed.

In simple terms: Imagine shaving off all buildings, trees, and objects - just the natural ground surface remains.

Uses: Understanding natural terrain, drainage, slopes for infrastructure planning.

Example: A DEM shows the hill shape, but not the houses built on top of it.

7. DSM (Digital Surface Model)

What it is: A grid of elevation values representing the TOP of everything you SEE - ground, buildings, trees, towers.

In simple terms: The visible surface. If you drop a ball, it hits the rooftop or treetop, not the ground.

Difference from DEM: DSM includes objects; DEM is bare earth only.

Example: DSM shows building rooftops at 20 meters; DEM shows ground at 2 meters.

8. nDSM (Normalized DSM)

What it is: Height of objects ABOVE the local ground. Calculated as: $nDSM = DSM - DEM$

In simple terms: Building height, tree height - measured from ground UP, not from sea level.

Example:

- Ground elevation (DEM) = 100m above sea level
- Building top (DSM) = 120m above sea level
- Building height (nDSM) = $120 - 100 = 20$ meters tall

Uses: Urban planning, forest height measurement, building height estimation.

9. rDSM (Relative DSM)

What it is: A surface where you know the SHAPE (this is higher than that) but NOT the absolute scale in meters.

In simple terms: You know rooftop is taller than road, but you don't know if it's 10 meters or 100 meters.

When it happens: Photo AI with single image produces rDSM - relative only, not metric.

How to fix: Calibration using reference data converts rDSM → metric DSM.

10. GEOREFERENCED

What it is: Every pixel in the image has a REAL geographic coordinate (latitude/longitude or projected coordinates) and a known pixel size.

In simple terms: The image knows WHERE it is on Earth. Pixel (0,0) = "I'm at latitude X, longitude Y, and I'm 10 meters wide."

Example: Non-georeferenced = random photo from your phone. Georeferenced = that photo tagged "taken at 28.6139°N, 77.2090°E with 0.5m resolution."

11. GeoTIFF

What it is: A TIFF image file that contains built-in geographic information - coordinates, projection system, pixel size, orientation.

In simple terms: A smart image that knows its place on Earth.

What it stores:

- Pixel values (colors or elevation numbers)
- CRS (coordinate system)
- Affine transform (6 numbers saying "pixel (0,0) starts here, each pixel is this wide")
- NoData value (like -9999 = "no measurement here")

Why crucial for TerraFly: Lets us align different images perfectly - optical image + DEM must cover the same geographic area.

12. SRTM (Shuttle Radar Topography Mission)

What it is: A FREE worldwide elevation dataset collected by NASA Space Shuttle in 2000. Approximately 30-meter resolution (90m in some regions).

In simple terms: Free DEM covering most of Earth - decent quality for terrain, not great for small details.

Accuracy: ~90% accuracy for terrain elevation. Good for mountains, valleys, general landscape.

Limitations: Captured in 2000, so new buildings/changes not included. Resolution too coarse for individual building heights.

TerraFly use: SRTM can be used as reference DEM for DEM Terrain workflow or for calibrating Photo AI.

13. GCP (Ground Control Point)

What it is: A specific point on the ground whose exact position (X, Y coordinates) and height (Z elevation) are ALREADY KNOWN from surveys.

In simple terms: Like a signpost saying "I am at exactly 28.6139°N, 77.2090°E, elevation 216.5 meters - surveyed and verified."

How it's measured: Surveyors use GPS, total stations, or precise instruments to measure GCP coordinates and elevation.

TerraFly use: We can use GCPs to calibrate Photo AI - "AI predicted height 0.8, but this GCP is really 25 meters, so we fit scale and offset."

14. SCALE CALIBRATION

What it is: Converting relative values (0 to 1) into real metric values (meters).

In simple terms: AI says "roof = 0.85, road = 0.15" (relative). Calibration converts to "roof = 18.4m, road = 2.1m" (meters).

The math:

- Find scale (multiplication factor) and offset (addition factor)
- $\text{metric_height} = (\text{relative_value} \times \text{scale}) + \text{offset}$

How we do it: Compare AI predictions to reference DSM or GCPs, fit the scale and offset that minimize error.

Gates: TerraFly only unlocks metric outputs if calibration passes strict tests (RMSE, correlation, inlier ratio).

15. 3D MESH

What it is: A surface made of many connected TRIANGLES. Each triangle corner (vertex) has X, Y coordinates and Z height.

In simple terms: Like a sheet of graph paper where each square is divided into triangles, and you raise each corner to its height value.

Why triangles: Only polygon that's ALWAYS flat and rigid. Any 3 points define a plane. Squares can twist.

How it's built:

1. Create a grid of points (vertices)
2. Connect them into triangles
3. Raise each vertex to its elevation value
4. Result = 3D terrain surface

In TerraFly: The viewer creates thousands of triangles to form the terrain surface.

16. TEXTURE

What it is: Placing the optical RGB image OVER the 3D mesh so colors from the photo appear on the 3D surface.

In simple terms: Like gift-wrapping a box - the flat wrapping paper (optical image) gets draped over the 3D shape (mesh).

Technical term: Texture mapping using UV coordinates.

Result: You see realistic 3D terrain with actual colors from the satellite photo.

17. FLYTHROUGH

What it is: Moving through the 3D scene like flying a drone or playing a video game.

In simple terms: Navigate around, zoom in/out, fly over mountains, orbit buildings.

TerraFly controls:

- **Orbit mode:** Left-drag rotates, right-drag pans, wheel zooms
 - **First-person mode:** WASD keys to move, mouse to look around, like a game
-

18. RMSE (Root Mean Square Error)

What it is: An error metric that STRONGLY PUNISHES large mistakes.

The math: Square each error, average them, take square root.

Why it matters: One huge error (predicted 10m, actual 50m = 40m error) affects RMSE much more than many small errors.

In simple terms: Strict accuracy test. Lower RMSE = better accuracy.

TerraFly use: We check RMSE on held-out points. Must be below threshold (like 5 meters) to pass calibration.

19. MAE (Mean Absolute Error)

What it is: Average of all errors in meters, without caring about direction.

The math: $|\text{predicted} - \text{actual}|$ for each point, then average.

In simple terms: On average, how many meters off are we?

Example: MAE = 3.2 meters means "on average, predictions are 3.2 meters away from truth."

Difference from RMSE: MAE treats all errors equally; RMSE punishes big errors more.

20. CORRELATION

What it is: Whether the predicted shape FOLLOWS the true shape - do high points match high, low match low?

Range: -1 to +1. Perfect correlation = 1.0, no relationship = 0, opposite = -1.

In simple terms: Does the AI surface look LIKE the real terrain, even if the scale isn't perfect?

Example: AI predicts relative values 0.2, 0.5, 0.9. True values are 10m, 25m, 45m. The SHAPE matches (both go up), so correlation is high.

TerraFly use: We require correlation > 0.7 for calibration to pass. Shape must follow reality.

21. PIXEL

What it is: One tiny square in a digital image - the smallest unit.

In simple terms: Like a tiny tile in a mosaic. Thousands of pixels make up one image.

Properties: Each pixel has a color (RGB values) or a number (elevation value in DEM).

22. RASTER

What it is: A grid of pixels. Images are rasters. DEMs are rasters. DSMs are rasters.

In simple terms: Organized rows and columns of values, like a spreadsheet where each cell has a number or color.

Opposite of raster: Vector (points, lines, polygons with exact coordinates).

23. RGB

What it is: Red, Green, Blue - the three color channels that combine to make any color.

Values: Each channel typically 0-255.

Example: Pure red = (255, 0, 0). White = (255, 255, 255). Black = (0, 0, 0).

24. CRS / EPSG

What it is: Coordinate Reference System - the "language" used to describe locations on Earth.

EPSG: A code identifying a specific CRS. Example: EPSG:4326 = WGS84 lat/lon, EPSG:32643 = UTM Zone 43N (India).

Why it matters: Two images must use the SAME CRS to align properly. TerraFly reprojects if needed.

25. RELATIVE DEPTH

What it is: Knowing the SHAPE (this higher than that) but NOT absolute meters.

In simple terms: You know order and proportions, but not scale.

When it happens: AI depth estimation from single image produces relative depth.

26. METRIC ELEVATION

What it is: Actual height values in METERS with a known reference (usually sea level).

In simple terms: Real numbers you can trust - "this point is 156.3 meters above sea level."

TerraFly requirement: Only show "metric" after calibration passes validation gates.

27. MODEL

What it is: A trained mathematical pattern-recognition system. Learned from thousands or millions of examples.

In simple terms: Like your brain learning to recognize faces. The model learned to recognize depth patterns.

Example: Depth Anything V2 model learned from millions of images with depth to predict depth from new images.

28. INFERENCE

What it is: USING an already-trained model to make predictions on new data.

In simple terms: Not training anymore - just running the model to get an output.

Example: You upload an image → model processes it → outputs depth map. That's inference.

29. TRAINING

What it is: Teaching a model by showing it many examples of input + correct output.

In simple terms: Like teaching a child by showing flashcards repeatedly until they learn the pattern.

Requirements: Paired data (input RGB + correct depth), compute power, time.

Why TerraFly didn't train: 4 days to build SYSTEM, not months to train model.

30. CALIBRATION

What it is: Converting relative AI predictions into metric values using independent reference data.

Process:

1. AI predicts relative surface
2. You provide reference DSM or GCPs (known heights)
3. System fits scale and offset

4. Validates on held-out points
5. If gates pass → unlock metric outputs

TerraFly honesty: Calibration can FAIL - we show why and stay in relative mode.

31. API (Application Programming Interface)

What it is: The agreement between frontend (what user sees) and backend (the processing code).

In simple terms: The "menu" of actions frontend can request from backend.

Example: Frontend says "POST /analyze with this image" → Backend responds "Job created, ID=12345, status=processing."

32. FRONTEND

What it is: The part of the application the USER SEES and CLICKS - buttons, upload areas, 3D viewer.

Technology in TerraFly: React + TypeScript + Three.js

Runs where: In your web browser at localhost:8000

33. BACKEND

What it is: The "brain" code that validates files, runs AI models, creates outputs, checks security.

Technology in TerraFly: FastAPI + Python

Runs where: On your computer as a local server

34. MESH (detailed)

What it is: 3D surface made from vertices (corner points) and faces (triangles connecting them).

Components:

- **Vertices:** Points with X, Y, Z coordinates
 - **Faces:** Triangles defined by 3 vertices
 - **Normals:** Direction each face points (for lighting)
-

35. GLB

What it is: A portable 3D file format (binary glTF) containing mesh, texture, materials, lighting info.

Why it's useful: Can open in Blender, Unity, web viewers - standard 3D exchange format.

What TerraFly exports: Mesh with embedded texture image, UV coordinates, normals, PBR materials.

36. HASH / SHA-256

What it is: A unique "fingerprint" of a file. Change one byte → completely different hash.

Uses: Prove a file hasn't been altered. Verify downloaded files match originals.

Example: SHA-256 of "hello" =

2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

TerraFly use: Every output artifact gets a hash recorded in the manifest.

37. MANIFEST

What it is: A detailed "receipt" file listing everything about a processing job.

Contains:

- Input filename and hash
- Model name and version
- Device used (CUDA or CPU)
- Processing time
- All output artifacts with hashes
- Warnings
- Calibration method (if used)

- Scientific state (Relative / Metric)

Why crucial: Makes results reproducible and auditable. Judges can verify claims.

Part 2: HOW THEY ALL WORK TOGETHER

THE BIG PICTURE: Two Workflows in TerraFly

WORKFLOW 1: DEM TERRAIN (How Components Connect)

Step 1: INPUT

You provide TWO GeoTIFF files:

- **Optical RGB GeoTIFF** (the color satellite image)
- **DEM GeoTIFF** (elevation data from SRTM or other source)

Step 2: VALIDATION

Backend checks:

- Are both files valid GeoTIFFs?
- Do they have CRS information?
- Do they cover the same geographic area (at least 90% overlap)?

Terms used: GeoTIFF, CRS/EPSG, Georeferenced, Raster

Step 3: ALIGNMENT

If CRS don't match or resolutions differ:

- Backend reprojects DEM to match optical image's CRS
- Resamples DEM to match optical pixel grid

Result: Both images are now pixel-perfect aligned - pixel (100, 200) in optical corresponds to pixel (100, 200) in DEM.

Terms used: CRS, EPSG, Pixel, GeoTIFF, Raster

Step 4: PRESERVE METRIC TRUTH

- Read DEM elevation values (in meters)
- Store as float32 array - preserves precision
- Keep NoData values marked (-9999 or specified value)
- Record vertical datum (sea level reference)

Terms used: DEM, Metric elevation, Elevation

Step 5: CREATE DISPLAY SURFACE

For visualization, normalize heights to 0-1 scale:

- Find min and max elevation
- Normalize: $(\text{value} - \text{min}) / (\text{max} - \text{min})$
- Apply vertical exaggeration (like $1.4\times$) for display ONLY
- Scientific arrays stay in meters

Terms used: Relative depth (for display only)

Step 6: BUILD 3D MESH

- Create grid of vertices matching pixel dimensions
- Each vertex position: (X_geo, Y_geo, Z_elevation)
- Connect vertices into triangles
- Calculate normals for lighting

Terms used: 3D mesh, Vertex, Triangle, Pixel

Step 7: APPLY TEXTURE

- Load optical RGB image
- Map image pixels to mesh triangles using UV coordinates
- Each triangle face gets colored by corresponding image pixels

Terms used: Texture, Texture mapping, RGB, Optical RGB image

Step 8: RENDER IN VIEWER

- Three.js loads mesh + texture
- Camera orbits or flies through scene
- Lighting simulates sun direction
- User clicks points → reads metric elevation values

Terms used: Flythrough, Frontend, 3D mesh

Step 9: EXPORT OUTPUTS

Backend creates:

- Metric GeoTIFF (elevation in meters with coordinates)
- GLB file (portable 3D with embedded texture)
- Alignment report (how reprojection was done)
- Provenance file (DEM source name, vertical datum)
- Manifest (all hashes, settings, timestamps)

Terms used: GeoTIFF, GLB, Hash/SHA-256, Manifest, Metric elevation

KEY INSIGHT FOR DEM TERRAIN:

Optical image provides COLOR → Texture

DEM provides HEIGHT → Mesh elevation

Alignment connects them → Georeferenced result

WORKFLOW 2: PHOTO AI (How Components Connect)

Step 1: INPUT

You provide ONE image:

- PNG, JPEG, or GeoTIFF
- Can be georeferenced or not

Step 2: VALIDATION

Backend checks:

- Not corrupt
- Size within limits
- Convert to 3-channel RGB if needed

Terms used: Optical RGB image, Pixel, RGB

Step 3: AI INFERENCE

- Load Depth Anything V2 model (pretrained)
- Run model on GPU (CUDA) or CPU
- For large images: split into overlapping tiles
- Model outputs: raw depth/proximity values for each pixel

Terms used: Model, Inference, Remote sensing (domain), Single-view

Step 4: CREATE RELATIVE SURFACE

- Raw output is inverse depth (closer = bigger number)
- Normalize to 0-1 scale globally
- Save as `relative_surface.npy` (scientific truth)
- This is rDSM - shape known, meters NOT known

Terms used: Depth map, Relative depth, rDSM, Raster

Step 5A: IF NO CALIBRATION

- Create display grid (smoothing, spike removal)
- Build 3D mesh using relative values
- Apply texture from input image
- Render in viewer
- Export: relative NPY, GLB, preview images
- State stays: "Relative - not metres"

Terms used: 3D mesh, Texture, GLB, Manifest

Step 5B: IF USER WANTS CALIBRATION

User provides reference data:

- **Option A:** Reference DSM covering same area
- **Option B:** Ground Control Points with surveyed elevations

Terms used: DSM, GCP, Georeferenced, Metric elevation

Step 6: ALIGNMENT (for reference DSM)

- Check CRS match between input and reference
- Reproject if needed
- Ensure pixel grids align

Terms used: CRS, EPSG, GeoTIFF, Alignment

Step 7: FIT SCALE AND OFFSET

- Compare AI relative values to reference metric values
- Use robust fitting (ignores outliers)
- Find: $\text{metric} = (\text{relative} \times \text{scale}) + \text{offset}$
- Split data: training points (fit) vs validation points (test)

Terms used: Scale calibration, Calibration, Training (fitting), GCP

Step 8: VALIDATION GATES

Check on held-out validation points:

- **RMSE** < user threshold (e.g., 5 meters)
- **Correlation** > 0.7 (shape follows reality)
- **MAE** reported
- Inlier ratio > 0.6 (enough good matches)
- Scale is positive (not inverted)

Terms used: RMSE, MAE, Correlation, Validation

Step 9A: IF GATES PASS

- State becomes: "Metric Calibrated"
- Convert relative surface to metric using scale/offset
- Create metric DSM (float32 in meters)
- Export metric GeoTIFF with CRS, affine, vertical datum
- Export residual map (errors at each pixel)
- Viewer now shows metric values when clicking
- Manifest records calibration method, reference source, gates

Terms used: DSM, Metric elevation, GeoTIFF, Hash, Manifest

Step 9B: IF GATES FAIL

- State stays: "Relative"
- Show WHY it failed (RMSE too high? Correlation low?)
- Keep relative outputs only

- No metric GeoTIFF created
- Manifest records rejection reason

Terms used: Relative depth, rDSM, Manifest

Step 10: VIEWER INTERACTION

User explores 3D scene:

- Orbit or first-person navigation
- Click points A and B
- If relative: show relative values (0.25, 0.78)
- If metric calibrated: show elevation (156m, 189m), height difference (33m), distance, slope

Terms used: Flythrough, Mesh, Texture, Elevation

KEY INSIGHT FOR PHOTO AI:

Single optical image → AI Model → Relative depth (rDSM)

Optional: + Reference data → Calibration → Metric DSM

Honesty: Only claim meters after validation passes

HOW PHOTO AI COMPONENTS INTERACT (Detailed Flow)

The AI Model Chain:

1. **Input:** Optical RGB image (single-view)
2. **Preprocessing:** Resize, normalize pixel values
3. **Model inference:** Depth Anything V2 processes patterns
4. **Output:** Depth map (relative, 0-1 values)
5. **Postprocessing:** Create relative surface array (rDSM)

Connection: Remote sensing image → Model → Depth map → Relative depth surface

The Calibration Chain:

1. **Input relative:** rDSM from AI
2. **Input reference:** DSM or GCPs (metric elevation)

3. **Alignment:** Match coordinates (CRS, georeferenced)
4. **Fitting:** Calculate scale calibration parameters
5. **Validation:** Test on held-out points (RMSE, MAE, Correlation)
6. **Gates:** Pass → Metric DSM; Fail → stay Relative
7. **Manifest:** Record everything (Hash, settings, results)

Connection: rDSM + DSM/GCP → Calibration → Validation → Metric DSM (if passed)

The 3D Visualization Chain:

1. **Height data:** Metric DSM or relative surface
2. **Color data:** Optical RGB image
3. **Mesh generation:** Create triangles (3D mesh), raise vertices
4. **Texture mapping:** Drape RGB image onto mesh
5. **Rendering:** Three.js draws in browser (Frontend)
6. **Interaction:** User flythrough, click for elevation
7. **Export:** GLB file (portable 3D)

Connection: DSM/rDSM + RGB → Mesh + Texture → GLB → Flythrough

COMPLETE SYSTEM FLOW DIAGRAM (In Words)

USER uploads image(s) ↓ [FRONTEND (React)] validates file, sends to API ↓ [BACKEND (FastAPI)] receives request ↓ Path splits: └─ DEM TERRAIN: Optical GeoTIFF + DEM GeoTIFF | ↓ | Check CRS, reproject if needed | ↓ | Align pixel grids (Georeferenced) | ↓ | Preserve metric elevation values | ↓ | Create 3D mesh from DEM | ↓ | Apply optical texture | ↓ | Export: Metric GeoTIFF, GLB, Manifest (with Hash) | └─ PHOTO AI: Single optical image ↓ Preprocess (RGB normalization) ↓ Run Model inference (Depth Anything V2) ↓ Create Depth map → rDSM (Relative depth) ↓ [Optional Calibration Path] ↓ User provides Reference DSM or GCPs? ↓ YES → Align (CRS), Fit scale calibration ↓ Validate on held-out points ↓ Check gates: RMSE, MAE, Correlation ↓ └─ PASS → Metric DSM (elevation in meters) | Export: Metric GeoTIFF, residuals, GLB | State: "Metric Calibrated" | └─ FAIL → Stay rDSM (relative only) Export: Relative NPY, GLB State: "Relative" ↓ NO → Export relative outputs only ↓ Create Manifest (Hash all artifacts) ↓ [BACKEND]

returns job ID, artifacts ↓ [FRONTEND] loads results ↓ Three.js renders 3D mesh with texture ↓ User experiences Flythrough ↓ User clicks points → Frontend reads elevation ↓ Displays: Relative values OR Metric elevation (depends on state)

THE GEOTIFF FRAMEWORK - THE GLUE BETWEEN COMPONENTS

What GeoTIFF Does:

1. **Stores pixel data:** Colors (RGB) or numbers (elevation)
2. **Embeds geographic info:** CRS, affine transform, pixel size
3. **Links raster to Earth:** Every pixel knows its real location

How TerraFly Uses It:

Reading inputs:

,

Optical GeoTIFF:

- Read RGB pixel values
- Extract: CRS = EPSG:32643, width = 1000, height = 800
- Extract: Affine transform = [10, 0, 280000, 0, -10, 3200000]
(Pixel size = 10m, top-left corner at 280000E, 3200000N)

DEM GeoTIFF:

- Read elevation pixel values (in meters)
- Extract: Same CRS or different? Same area?
- Extract: Pixel size, bounds

,

Aligning:

,

If DEM is 20m resolution and optical is 10m:

- Resample DEM from 500×400 to 1000×800
- Use bilinear interpolation

- Now both grids match perfectly

Exporting metric results:

Create new GeoTIFF:

- Write calibrated elevation values (float32)
- Embed same CRS as input
- Embed same affine transform
- Add vertical datum tag (e.g., EGM96)
- Add NoData value (-9999)

Result: Output GeoTIFF can be opened in GIS software (QGIS, ArcGIS), overlaid with other data, analyzed - because it's georeferenced.

WHY THE TWO-PATH DESIGN IS SMART

DEM Terrain Path (Production):

- **Needs:** Optical image + existing DEM
- **Provides:** Metric elevation (trustworthy meters)
- **Limitation:** Requires pre-existing elevation data
- **Best for:** Mountains, terrain, surveyed areas

Components used: GeoTIFF, DEM, CRS, Alignment, Mesh, Texture, GLB, Manifest

Photo AI Path (Experimental):

- **Needs:** Single optical image only
- **Provides:** Relative depth OR metric (if calibration passes)
- **Limitation:** Domain shift, needs training for accuracy
- **Best for:** Areas without DEM, future research direction

Components used: Model, Inference, Depth map, rDSM, Calibration, GCP, DSM, RMSE, Correlation, Manifest

Together they cover:

- Areas WITH elevation data → DEM Terrain (reliable NOW)
 - Areas WITHOUT elevation data → Photo AI framework (future improvement)
-

FINAL SUMMARY: THE COMPLETE PICTURE

Starting Point:

- Remote sensing captures optical RGB images from single-view

Problem:

- Images are flat (2D raster of pixels)
- Need 3D with elevation

Solution 1 - DEM Terrain:

Optical GeoTIFF + DEM GeoTIFF

- Check CRS compatibility
- Align using georeferenced coordinates
- Preserve metric elevation from DEM
- Build 3D mesh with metric heights
- Apply texture from optical image
- Export GLB, metric GeoTIFF with hash in manifest
- User flythrough with real elevations

Solution 2 - Photo AI:

Single optical image

- AI model inference produces depth map
- Creates rDSM (relative depth)
- Optional: Add reference DSM or GCPs for scale calibration
- Validation gates check RMSE, MAE, Correlation
- If pass: Metric DSM (elevation in meters)
- If fail: Stay relative
- Build mesh (metric or relative)
- Apply texture
- Export GLB, GeoTIFF (if metric), manifest with hash
- User flythrough with appropriate values

Key Components Working Together:

- **GeoTIFF framework:** Aligns optical + elevation data using CRS

- **Mesh + Texture:** Combines height (DSM/rDSM) with color (RGB)
- **Calibration:** Converts relative → metric using scale fitting
- **Validation:** RMSE, MAE, Correlation ensure quality
- **Manifest + Hash:** Makes results reproducible and auditable
- **Frontend + Backend + API:** User interaction → processing → visualization

Honest Design:

- DEM Terrain: Reports source DEM accuracy (SRTM ~90%)
 - Photo AI: Only claims metric after validation passes gates
 - If uncertain: Stay in relative mode with clear warning
 - Every output: Hashed, recorded, traceable
-

FOR YOUR TEAM TOMORROW

When explaining to teammates, use this flow:

1. **Show optical image:** "This is our color photo - flat, no height"
2. **Show DEM:** "This is elevation data - heights in meters"
3. **Explain alignment:** "GeoTIFF lets us match them pixel-by-pixel"
4. **Show mesh:** "We build triangles and raise them to DEM heights"
5. **Show texture:** "We drape the photo over the triangles"
6. **Show result:** "Now you can fly through realistic 3D"

For Photo AI:

1. **One image in:** "AI looks at patterns and GUESSES depth"
2. **Relative out:** "Shape is right, but not meters yet"
3. **Calibration:** "We compare to known heights (GCP/DSM) and fit scale"
4. **Validation:** "We test on separate points - RMSE, correlation"
5. **Gates:** "Only if it passes do we say 'metric' - otherwise honest 'relative'"

Key message: Every term connects to create one honest, traceable, working 3D terrain system.

END OF GUIDE